

class FormFieldsContainer

- class FormFieldsContainer
 - Construtor **constructor**
 - Descrição
 - Propriedades
 - Exemplo de Uso
 - Método **getTextWidth**
 - Descrição
 - Parâmetros
 - Retorno
 - Exemplo de Uso
 - Método **addFieldRow**
 - Descrição
 - Parâmetros
 - Estrutura do Campo
 - Retorno
 - Exemplo de Uso
 - Método **adjustColumnWidths**
 - Descrição
 - Funcionamento Interno
 - Parâmetros
 - Retorno
 - Exemplo de Uso
 - Método **getElement**
 - Descrição
 - Parâmetros
 - Retorno
 - Exemplo de Uso
 - REFERÊNCIAS

Construtor **constructor**

Cria uma nova instância da classe com um elemento contêiner **div** para campos de formulário em um diálogo de entrada.

Descrição

- O construtor inicializa o contêiner principal do formulário, define uma classe CSS para estilização e calcula a largura dos caracteres com base em uma fonte de 12 pixels.

Propriedades

- **container**: Um elemento **div** criado dinamicamente para encapsular os campos de formulário. Este **div** possui a classe CSS **inputBox-dialog-form-fields**, que pode ser estilizada para definir o layout e aparência dos campos de formulário.
- **fontSize**: Define o tamanho da fonte em pixels. O valor padrão é **12px**.
- **charWidth**: Representa a largura média de um caractere, estimada em 60% do tamanho da fonte definida (no caso, **fontSize * 0.6**), facilitando o cálculo dinâmico de larguras de campos baseadas no número de caracteres.

Exemplo de Uso

```
const formFieldsContainer = new FormFieldsContainer();
document.body.appendChild(formFieldsContainer.container);
```

Método **getTextWidth**

Calcula a largura de um texto em pixels, com base em um tamanho de fonte específico.

Descrição

O método **getTextWidth** utiliza o contexto de desenho (**2d context**) de um elemento **canvas** temporário para calcular a largura de um texto especificado. Ele permite estimar dinamicamente o espaço necessário para exibir um texto em uma fonte e tamanho definidos.

Parâmetros

- **text**: (string) O texto cujo comprimento será medido.
- **fontSize**: (number) O tamanho da fonte em pixels a ser usado na medição.

Retorno

- **(number)**: A largura do texto em pixels, medida com a fonte especificada.

Exemplo de Uso

```
const width = getTextWidth("Exemplo de texto", 16);
console.log(`A largura do texto é: ${width}px`);
```

Método **addFieldRow**

Adiciona uma linha de campos ao formulário, com base nas especificações de cada campo fornecidas.

Descrição

O método `addFieldRow` cria uma nova linha de entrada (`fieldsetElement`) no contêiner principal do formulário. Para cada campo especificado na `line`, ele cria um rótulo (`label`) e um campo de entrada (`input`), aplicando as classes CSS e configurações de estilo necessárias.

Parâmetros

- **line**: (array) Uma lista de objetos que representam os campos a serem adicionados na linha do formulário. Cada objeto deve conter:
 - **label**: (string) O texto a ser exibido no rótulo do campo.
 - **align**: (string) A orientação do rótulo ('right' ou 'left'), controlada por classes CSS `align-right` e `align-left`.
 - **maxLength**: (number) O comprimento máximo do campo, usado para definir a largura do campo de entrada.

Estrutura do Campo

Para cada item em `line`, o método realiza os seguintes passos:

1. **Contêiner do Campo**: Cria um elemento `div` para encapsular o rótulo e o campo de entrada.
2. **Rótulo (`labelElement`)**: Define o rótulo com o texto especificado e aplica uma classe CSS de alinhamento (`align-right` ou `align-left`).
3. **Campo de Entrada (`inputField`)**: Cria um campo de entrada do tipo texto (`type='text'`) com comprimento máximo (`maxLength`).
 - **Campo Oculto**: Se o `label` for vazio e `maxLength` for 0, aplica a classe `hidden` ao campo de entrada e ao rótulo, tornando-os invisíveis.
 - **Largura do Campo**: Para campos visíveis, calcula a largura do campo com base no `maxLength` e no `charWidth`, que estima a largura de cada caractere.

Retorno

Nenhum valor de retorno. O método modifica o DOM adicionando uma nova linha de campos ao contêiner principal (`this.container`).

Exemplo de Uso

```
const fields = [  
  { label: 'Nome', align: 'left', maxLength: 20 },  
  { label: 'Idade', align: 'right', maxLength: 3 },  
  { label: '', maxLength: 0 } // Campo invisível  
];  
formDialog.addFieldRow(fields);
```

Método `adjustColumnWidths`

Ajusta a largura das colunas dos rótulos (**label**) e dos campos de entrada (**input**) no formulário para garantir alinhamento consistente, com base na largura máxima necessária em cada coluna.

Descrição

O método **adjustColumnWidths** calcula a largura máxima de cada coluna de elementos **label** e **input** em todas as linhas do formulário e aplica esses valores, garantindo que todos os elementos de uma mesma coluna fiquem alinhados.

Funcionamento Interno

1. **Seleção das Linhas:** Obtém todas as linhas do formulário usando a classe **.inputBox-dialog-form-row**.
2. **Cálculo das Larguras:** Para cada linha, percorre os elementos **label** e **input** e determina a largura máxima necessária para cada coluna.
 - Para os rótulos, utiliza o método **getTextWidth** para calcular a largura do texto.
 - Para os campos de entrada, utiliza **offsetWidth** para obter a largura real do campo.
3. **Armazenamento das Larguras:** As larguras máximas são armazenadas no array **columnWidths**, onde cada índice representa uma coluna.
4. **Aplicação das Larguras:** Após determinar as larguras, aplica os valores aos elementos **label** e **input** de cada linha, ajustando a largura com **style.width**.

Parâmetros

Este método não possui parâmetros. Ele opera diretamente nos elementos do contêiner **this.container**.

Retorno

Nenhum valor de retorno. Este método ajusta os estilos dos elementos **label** e **input** no DOM.

Exemplo de Uso

```
formDialog.adjustColumnWidths();
```

Método **getElement**

Retorna o elemento contêiner principal do formulário, permitindo acesso direto ao elemento DOM onde o formulário e seus campos são gerenciados.

Descrição

O método `getElement` é uma função simples que retorna o elemento contêiner principal (`this.container`) utilizado pela classe para armazenar a estrutura do formulário. Esse contêiner contém todas as linhas e campos de entrada adicionados ao formulário.

Parâmetros

Este método não possui parâmetros.

Retorno

- **Elemento DOM (`this.container`):** O elemento `div` que representa o contêiner principal do formulário, contendo todas as linhas e campos de entrada.

Exemplo de Uso

Este método pode ser utilizado para acessar e manipular o contêiner principal do formulário diretamente no DOM.

```
const formContainer = formDialog.getElement();
document.body.appendChild(formContainer); // Adiciona o formulário ao
corpo do documento
```

REFERÊNCIAS

1. [Chatgpt](#)
2. [Código fonte](#)